

종합실습 1

학사관리시스템(University DB) 설계 및 구현 리포트

이름 : 허서연

제출일 : 2026-07-22

PostgreSQL 기반 개념·논리·물리 데이터 모델링부터 DDL/DML, 필수 쿼리, 보너스 쿼리(Window Function, GROUP BY+HAVING), 제약조건(FK 옵션) 동작 검증까지의 전체 과정을 정리한 문서입니다.

1. 요구사항 및 설계 방향

1-1. 학사관리시스템 요구사항

대학의 학사 업무 중 핵심 흐름인 "학과 운영 → 교수 배정 → 강좌 개설 → 학생 수강신청 → 성적 관리"를 지원하는 데이터베이스를 구축한다. 시스템은 학과(departments), 교수(professors), 학생(students), 강좌(courses), 수강신청(enrollments) 5개 핵심 테이블을 관리하며, 각 테이블 간 참조 무결성(FK 제약조건)을 보장한다.

1-2. 설계 방향 — FK 제약조건 옵션의 의도적 다양화

이 설계에서는 부모-자식 관계마다 서로 다른 FK 삭제 옵션(ON DELETE)을 적용하여, 실제 업무 상황에 맞는 데이터 보존/삭제 정책을 구현했다.

- **professors.department_id → departments.id : SET NULL** — 학과가 없어져도 교수 기록은 보존되어야 한다
- **courses.professor_id → professors.id : SET NULL** — 담당교수가 퇴사해도 강좌 개설 기록은 유지되어야 한다
- **students.department_id → departments.id : RESTRICT** — 재학생이 있는 학과는 임의로 삭제할 수 없어야 한다
- **enrollments(student_id, course_id) → students/courses.id : CASCADE** — 수강신청은 학생·강좌 중 하나라도 없어지면 함께 삭제되는 약한 엔티티다

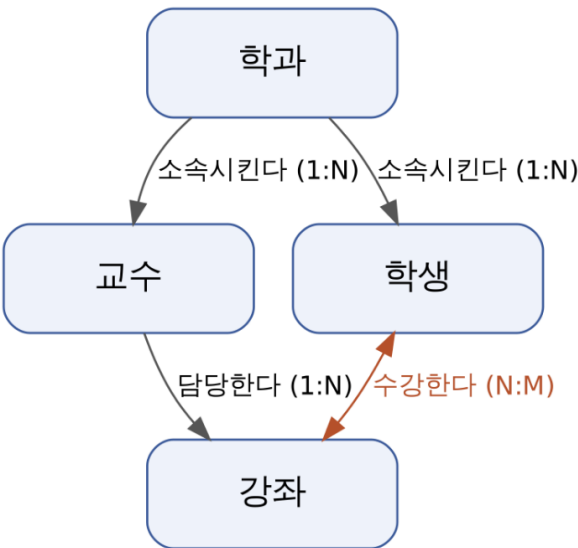
이렇게 세 가지 FK 옵션(SET NULL / RESTRICT / CASCADE)을 모두 사용함으로써, 각 옵션이 실제로 어떻게 동작하는지 8장에서 DELETE 명령으로 직접 검증한다.

2. 데이터 모델링 — 개념적 → 논리적 → 물리적

데이터베이스 설계는 추상적인 개념에서 시작해 점차 구체적인 구현으로 좁혀지는 3단계 과정을 거쳤다.

2-1. 개념적 모델링

이 단계에서는 타입, 제약조건, PK/FK 표기 없이 "어떤 개체들이 있고 서로 어떻게 얹혀있는가"만 표현한다. 학생과 강좌는 N:M 관계로, 아직 교차 테이블 분해는 하지 않은 상태다.



[그림 1] 개념적 모델 — 엔티티와 관계만 표현

2-2. 논리적 모델링

개념 모델에 PK/FK, 카디널리티, 정규화를 적용한 단계다. 특히 학생↔강좌의 N:M 관계는 논리 모델에서 enrollments 교차 테이블로 분해되었다.

- **1NF** — 모든 컬럼이 원자값. 예: email은 학생 1명당 1개만 저장(다중값 없음)
- **2NF** — enrollments의 복합키(student_id, course_id)에 대해 score가 두 키 모두에 완전 종속(부분 종속 없음)
- **3NF** — students.department_id로 FK만 저장하고 학과명을 학생 테이블에 직접 중복 저장하지 않음(이행 종속 제거)

2-3. 물리적 모델링

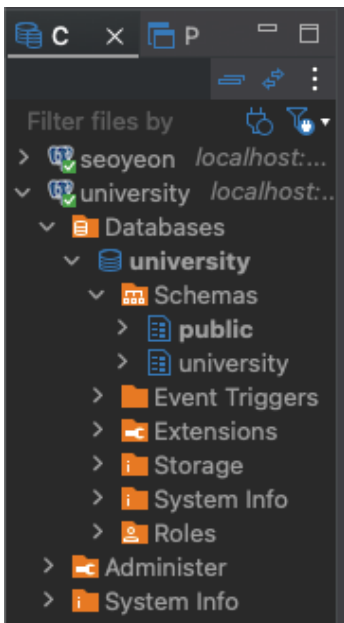
실제 PostgreSQL 타입과 제약조건을 확정하는 단계로, 아래와 같은 근거로 타입을 선정했다.

- 모든 PK → BIGINT GENERATED ALWAYS AS IDENTITY — 자연키 대신 대체키 사용
- email (students/professors) → VARCHAR(100) UNIQUE — 가변 길이 + 중복 불가
- students.grade → SMALLINT CHECK (1~4) — 학년은 1~4 범위로 고정된 도메인
- courses.credit → SMALLINT CHECK (>0) — 학점은 항상 양수
- enrollments.score → NUMERIC(5,2) CHECK (0~100) — 부동소수점 오차 방지, 소수점 포함 정확한 값 필요
- hired_date / birth_date → DATE — 시각(시분초) 불필요, 날짜만 필요
- enrollments.enrolled_at → TIMESTAMPTZ DEFAULT now() — 신청 시각은 타임존 포함 기록이 원칙

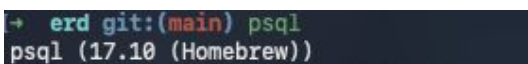
3. PostgreSQL 환경 구성

3-1. 접속 확인

DBeaver 접속 화면과 터미널의 psql 버전 확인 결과다.



[그림 2] DBeaver 접속 화면



[그림 3] psql 버전 확인

3-2. 데이터베이스 및 스키마 생성

DB명과 스키마명을 모두 university로 통일하여 생성했다. PostgreSQL에서는 데이터베이스와 스키마가 서로 다른 네임스페이스이므로 이름이 같아도 충돌하지 않는다.

```
CREATE DATABASE university
WITH ENCODING = 'UTF8'
LC_COLLATE = 'en_US.UTF-8'
LC_CTYPE = 'en_US.UTF-8'
TEMPLATE = template0;
```

```
seoyeon=# CREATE DATABASE university
WITH ENCODING = 'UTF8'
LC_COLLATE = 'en_US.UTF-8'
LC_CTYPE = 'en_US.UTF-8'
| TEMPLATE = template0;
CREATE DATABASE
```

[그림 4] CREATE DATABASE 실행 결과

```
[seoyeon=# psql: \c university
접속 정보 : 데이터베이스="university", 사용자="seoyeon".
```

[그림 5] university 데이터베이스 접속 확인

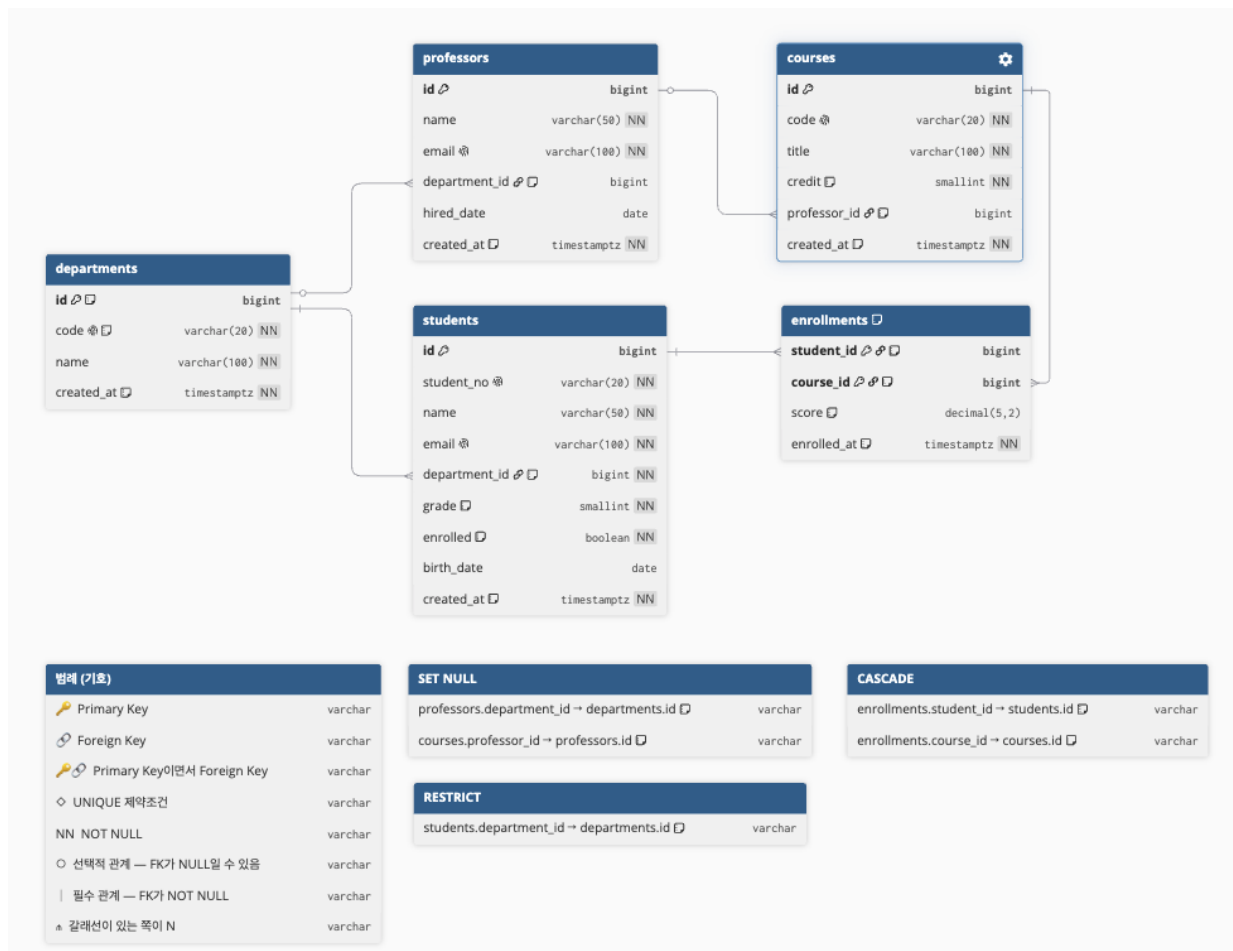
```
CREATE SCHEMA university;
SET search_path = university, public;
SELECT schema_name FROM information_schema.schemata
WHERE schema_name = 'university';
```

```
(university=# CREATE SCHEMA university;
CREATE SCHEMA
(university=# SET search_path = university, public;
SET
(university=# SELECT schema_name FROM information_schema.schemata WHERE schema_name = 'university';
 schema_name
-----
university
(1개 행)
```

[그림 6] CREATE SCHEMA 및 검증 결과

4 ERD (Entity-Relationship Diagram)

DDL을 작성하기 전에 dbdiagram.io로 논리 모델 ERD를 먼저 그려 관계와 제약조건을 시각적으로 검토했다. 범례를 4개의 표로 구성해 기호 의미와 함께, FK 삭제 옵션(SET NULL / RESTRICT / CASCADE)별로 어떤 관계가 여기에 해당하는지 명시적으로 그룹화했다.



[그림 7] dbdiagram.io 논리 모델 ERD 및 범례

5. DDL 작성 — 5 개 테이블

FK가 참조하는 부모 테이블을 먼저 생성하는 순서(departments → professors/students → courses → enrollments)로 작성했다.

5-1. departments (학과) — 강한 엔티티, 가장 먼저 생성

```
university=# CREATE TABLE departments (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  code        VARCHAR(20) NOT NULL UNIQUE,
  name        VARCHAR(100) NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE
```

[그림 8] departments 테이블 생성

5-2. professors (교수) — SET NULL

학과가 삭제되어도 교수 레코드는 유지되도록 department_id에 ON DELETE SET NULL을 적용했다.

```
university=# CREATE TABLE professors (  
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  name        VARCHAR(50) NOT NULL,  
  email       VARCHAR(100) NOT NULL UNIQUE,  
  department_id BIGINT REFERENCES departments(id) ON DELETE SET NULL,  
  hired_date  DATE,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE TABLE
```

[그림 9] professors 테이블 생성

5-3. students (학생) — RESTRICT

재학생이 있는 학과는 삭제할 수 없도록 department_id에 ON DELETE RESTRICT를 적용했다. grade는 CHECK로 1~4 범위만 허용한다.

```
university=# CREATE TABLE students (  
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  student_no  VARCHAR(20) NOT NULL UNIQUE,  
  name        VARCHAR(50) NOT NULL,  
  email       VARCHAR(100) NOT NULL UNIQUE,  
  department_id BIGINT NOT NULL REFERENCES departments(id) ON DELETE RESTRICT,  
  grade       SMALLINT NOT NULL CHECK (grade BETWEEN 1 AND 4),  
  enrolled    BOOLEAN NOT NULL DEFAULT TRUE,  
  birth_date  DATE,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE TABLE
```

[그림 10] students 테이블 생성

5-4. courses (강좌) — SET NULL

담당교수가 퇴사해도 강좌 기록이 남도록 professor_id에 ON DELETE SET NULL을 적용했다.

```
university=# CREATE TABLE courses (  
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  code        VARCHAR(20) NOT NULL UNIQUE,  
  title       VARCHAR(100) NOT NULL,  
  credit      SMALLINT NOT NULL CHECK (credit > 0),  
  professor_id BIGINT REFERENCES professors(id) ON DELETE SET NULL,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE TABLE
```

[그림 11] courses 테이블 생성

5-5. enrollments (수강신청) — 약한 엔티티, 복합 PK, CASCADE

student_id와 course_id 두 FK를 묶어 복합 기본키로 사용하는 약한 엔티티다. 둘 다 ON DELETE CASCADE를 적용해 부모(학생 또는 강좌)가 삭제되면 수강신청 기록도 함께 삭제된다.

```

university=# CREATE TABLE enrollments (
  student_id BIGINT NOT NULL REFERENCES students(id) ON DELETE CASCADE,
  course_id BIGINT NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
  score NUMERIC(5,2) CHECK (score BETWEEN 0 AND 100),
  enrolled_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (student_id, course_id)
);
CREATE TABLE

```

[그림 12] enrollments 테이블 생성

5-6. 생성 결과 확인

```

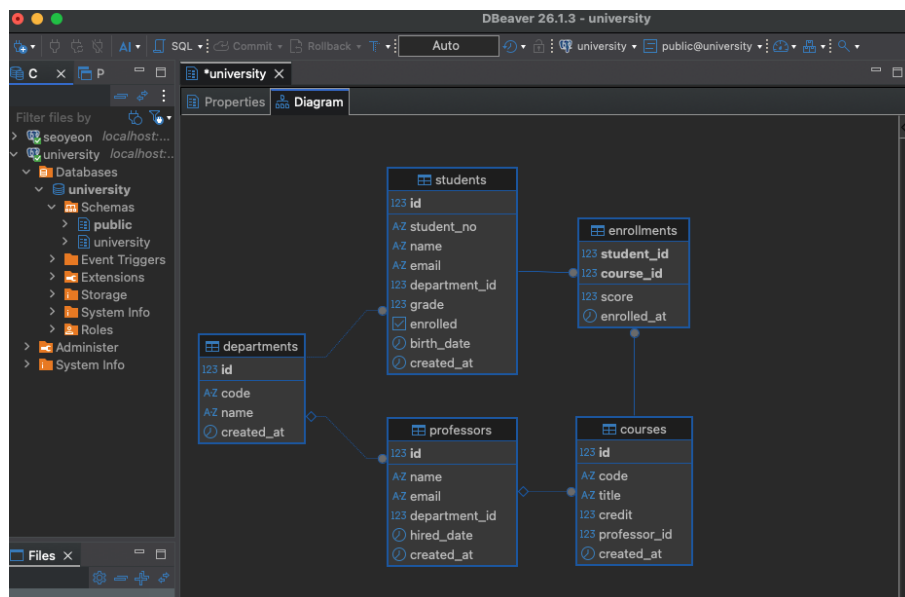
[university=# \dt university.*
릴레이션 목록
스키마 | 이름 | 형태 | 소유주
-----+-----+-----+-----
university | courses | 테이블 | seoyeon
university | departments | 테이블 | seoyeon
university | enrollments | 테이블 | seoyeon
university | professors | 테이블 | seoyeon
university | students | 테이블 | seoyeon
(5개 행)

```

[그림 13-1] \dt university.* 실행 결과 — 5개 테이블 모두 생성 확인

5-6. PostgreSQL 접속 결과 화면

PostgreSQL 스키마를 DBeaver에서 자동으로 읽어와 ERD를 생성했다. 논리/물리 모델링 단계에서 세운 설계가 실제 구현과 정확히 일치함을 시각적으로 재확인할 수 있다.



[그림 13-2] DBeaver 자동 생성 ERD — 설계와 실제 PostgreSQL 구현과 일치하는지 확인

6. DML — 샘플 데이터 삽입

모든 테이블에 최소 10건 이상의 데이터를 삽입했다. ID는 GENERATED ALWAYS AS IDENTITY라 직접 지정할 수 없으므로, 서브쿼리로 자연키(code/email/student_no)를 조회해 FK 값을 채우는 방식을 사용했다. 또한 이후 쿼리 실습에서 활용할 시나리오를 의도적으로 심어두었다: 휴학생(enrolled=false) 1명, 담당교수 미배정 강좌(professor_id=NULL) 1건, 성적 미입력(score=NULL) 2건, 수강신청이 전혀 없는 학생 1명.

6-1. departments (10 건)

```
university=# INSERT INTO departments (code, name) VALUES
('CSE', '컴퓨터공학과'),
('EE', '전자공학과'),
('ME', '기계공학과'),
('CHE', '화학공학과'),
('IE', '산업공학과'),
('BIZ', '경영학과'),
('ECON', '경제학과'),
('ENG', '영어영문학과'),
('PSY', '심리학과'),
('MATH', '수학과');
INSERT 0 10
```

[그림 14] departments INSERT 결과

6-2. professors (10 건)

```
university=# INSERT INTO professors (name, email, department_id, hired_date) VALUES
('김민준', 'minjun.kim@univ.ac.kr', (SELECT id FROM departments WHERE code='CSE'), '2015-03-01'),
('이서연', 'seoyeon.lee@univ.ac.kr', (SELECT id FROM departments WHERE code='CSE'), '2018-09-01'),
('박지훈', 'jihoon.park@univ.ac.kr', (SELECT id FROM departments WHERE code='EE'), '2012-03-01'),
('최유진', 'yujin.choi@univ.ac.kr', (SELECT id FROM departments WHERE code='ME'), '2016-03-01'),
('정하은', 'haeun.jung@univ.ac.kr', (SELECT id FROM departments WHERE code='CHE'), '2019-03-01'),
('강도윤', 'doyoon.kang@univ.ac.kr', (SELECT id FROM departments WHERE code='IE'), '2014-09-01'),
('윤서아', 'seoa.yoon@univ.ac.kr', (SELECT id FROM departments WHERE code='BIZ'), '2011-03-01'),
('임재현', 'jaehyun.lim@univ.ac.kr', (SELECT id FROM departments WHERE code='ECON'), '2017-03-01'),
('한소율', 'soyul.han@univ.ac.kr', (SELECT id FROM departments WHERE code='ENG'), '2013-03-01'),
('오민서', 'minseo.oh@univ.ac.kr', (SELECT id FROM departments WHERE code='MATH'), '2020-03-01');
INSERT 0 10
```

[그림 15] professors INSERT 결과

6-3. students (12 건, 휴학생 1 건 포함)

```

university=# INSERT INTO students (student_no, name, email, department_id, grade, enrolled, birth_date) VALUES
('2021001', '김하늘', 'haneul.kim@student.ac.kr', (SELECT id FROM departments WHERE code='CSE'), 4, TRUE, '2002-03-15'),
('2021002', '이도현', 'dohyun.lee@student.ac.kr', (SELECT id FROM departments WHERE code='CSE'), 4, TRUE, '2002-07-22'),
('2022001', '박서준', 'seojun.park@student.ac.kr', (SELECT id FROM departments WHERE code='CSE'), 3, TRUE, '2003-01-10'),
('2022002', '최지우', 'jiwoo.choi@student.ac.kr', (SELECT id FROM departments WHERE code='EE'), 3, TRUE, '2003-05-30'),
('2022003', '정예은', 'yeeun.jung@student.ac.kr', (SELECT id FROM departments WHERE code='ME'), 3, TRUE, '2003-11-02'),
('2023001', '강민재', 'minjae.kang@student.ac.kr', (SELECT id FROM departments WHERE code='CHE'), 2, TRUE, '2004-02-18'),
('2023002', '윤아린', 'arin.yoon@student.ac.kr', (SELECT id FROM departments WHERE code='IE'), 2, TRUE, '2004-09-09'),
('2023003', '임서윤', 'seoyun.lim@student.ac.kr', (SELECT id FROM departments WHERE code='BIZ'), 2, FALSE, '2004-04-25'),
('2024001', '한지호', 'jiho.han@student.ac.kr', (SELECT id FROM departments WHERE code='ECON'), 1, TRUE, '2005-06-14'),
('2024002', '오채원', 'chaewon.oh@student.ac.kr', (SELECT id FROM departments WHERE code='ENG'), 1, TRUE, '2005-08-08'),
('2024003', '김도윤', 'doyoon.kim@student.ac.kr', (SELECT id FROM departments WHERE code='PSY'), 1, TRUE, '2005-12-01'),
('2024004', '이수아', 'sua.lee@student.ac.kr', (SELECT id FROM departments WHERE code='MATH'), 1, TRUE, '2005-03-27');
INSERT 0 12

```

[그림 16] students INSERT 결과

6-4. courses (12 건, 담당교수 미배정 1 건 포함)

```

university=# INSERT INTO courses (code, title, credit, professor_id) VALUES
('CSE201', '자료구조', 3, (SELECT id FROM professors WHERE email='minjun.kim@univ.ac.kr')),
('CSE202', '데이터베이스', 3, (SELECT id FROM professors WHERE email='seoyeon.lee@univ.ac.kr')),
('CSE301', '운영체제', 3, (SELECT id FROM professors WHERE email='minjun.kim@univ.ac.kr')),
('EE201', '회로이론', 3, (SELECT id FROM professors WHERE email='jihoon.park@univ.ac.kr')),
('ME201', '열역학', 3, (SELECT id FROM professors WHERE email='yujin.choi@univ.ac.kr')),
('CHE201', '유기화학', 3, (SELECT id FROM professors WHERE email='haeun.jung@univ.ac.kr')),
('IE201', '생산관리', 2, (SELECT id FROM professors WHERE email='doyoon.kang@univ.ac.kr')),
('BIZ101', '경영학원론', 3, (SELECT id FROM professors WHERE email='seoa.yoon@univ.ac.kr')),
('ECON101', '미시경제학', 3, (SELECT id FROM professors WHERE email='jaehyun.lim@univ.ac.kr')),
('ENG101', '영어작문', 2, (SELECT id FROM professors WHERE email='soyul.han@univ.ac.kr')),
('MATH101', '미적분학', 3, (SELECT id FROM professors WHERE email='minseo.oh@univ.ac.kr')),
('PSY101', '심리학개론', 3, NULL);
INSERT 0 12

```

[그림 17] courses INSERT 결과

6-5. enrollments (14 건, 성적 미입력 2 건 포함)

```

university=# INSERT INTO enrollments (student_id, course_id, score) VALUES
((SELECT id FROM students WHERE student_no='2021001'), (SELECT id FROM courses WHERE code='CSE201'), 95),
((SELECT id FROM students WHERE student_no='2021001'), (SELECT id FROM courses WHERE code='CSE202'), 88),
((SELECT id FROM students WHERE student_no='2021002'), (SELECT id FROM courses WHERE code='CSE202'), 92),
((SELECT id FROM students WHERE student_no='2021002'), (SELECT id FROM courses WHERE code='CSE301'), 84),
((SELECT id FROM students WHERE student_no='2022001'), (SELECT id FROM courses WHERE code='CSE201'), 78),
((SELECT id FROM students WHERE student_no='2022001'), (SELECT id FROM courses WHERE code='CSE301'), NULL),
((SELECT id FROM students WHERE student_no='2022002'), (SELECT id FROM courses WHERE code='EE201'), 85),
((SELECT id FROM students WHERE student_no='2022003'), (SELECT id FROM courses WHERE code='ME201'), 90),
((SELECT id FROM students WHERE student_no='2023001'), (SELECT id FROM courses WHERE code='CHE201'), 82),
((SELECT id FROM students WHERE student_no='2023002'), (SELECT id FROM courses WHERE code='IE201'), 76),
((SELECT id FROM students WHERE student_no='2024001'), (SELECT id FROM courses WHERE code='ECON101'), 88),
((SELECT id FROM students WHERE student_no='2024002'), (SELECT id FROM courses WHERE code='ENG101'), 91),
((SELECT id FROM students WHERE student_no='2024003'), (SELECT id FROM courses WHERE code='PSY101'), NULL),
((SELECT id FROM students WHERE student_no='2024004'), (SELECT id FROM courses WHERE code='MATH101'), 95);
INSERT 0 14

```

[그림 18] enrollments INSERT 결과

6-6. 전체 건수 확인

```

university=# SELECT 'departments' AS tbl, COUNT(*) FROM departments
UNION ALL SELECT 'professors', COUNT(*) FROM professors
UNION ALL SELECT 'students', COUNT(*) FROM students
UNION ALL SELECT 'courses', COUNT(*) FROM courses
UNION ALL SELECT 'enrollments', COUNT(*) FROM enrollments;

```

tbl	count
departments	10
professors	10
students	12
courses	12
enrollments	14

(5개 행)

[그림 19] 테이블별 건수 확인 (UNION ALL 활용)

7. 필수 쿼리 실습

문항 1 — SELECT + WHERE + ORDER BY 기초 조회

1-1. 재학 중인 컴퓨터공학과 학생을 학년 내림차순으로 조회

```

university=# SELECT s.student_no, s.name, s.grade, s.email
FROM students s
JOIN departments d ON d.id = s.department_id
WHERE d.code = 'CSE' AND s.enrolled = TRUE
ORDER BY s.grade DESC;

```

student_no	name	grade	email
2021001	김 하 늘	4	haneul.kim@student.ac.kr
2021002	이 도 현	4	dohyun.lee@student.ac.kr
2022001	박 서 준	3	seojun.park@student.ac.kr

(3개 행)

[그림 20] 1-1 실행 결과

1-2. 3 학점 이상 강좌를 학점 내림차순, 동일 학점이면 강좌명 오름차순으로 조회

```

university=# SELECT code, title, credit
FROM courses
WHERE credit >= 3
ORDER BY credit DESC, title ASC;

```

code	title	credit
BIZ101	경영 학원론	3
CSE202	데 이 터 베 이 스	3
ECON101	미 시 경 제 학	3
MATH101	미 적 분 학	3
PSY101	심 리 학 개 론	3
ME201	열 역 학	3
CSE301	운 영 체 제	3
CHE201	유 기 화 학	3
CSE201	자 료 구 조	3
EE201	회 로 이 론	3

(10개 행)

[그림 21] 1-2 실행 결과

1-3. 2003~2005 년 출생 학생 조회 (BETWEEN 활용)

```
university=# SELECT name, student_no, birth_date
FROM students
WHERE birth_date BETWEEN '2003-01-01' AND '2005-12-31'
ORDER BY birth_date;
```

name	student_no	birth_date
박 서 준	2022001	2003-01-10
최 지 우	2022002	2003-05-30
정 예 은	2022003	2003-11-02
강 민 재	2023001	2004-02-18
임 서 윤	2023003	2004-04-25
윤 아 린	2023002	2004-09-09
이 수 아	2024004	2005-03-27
한 지 호	2024001	2005-06-14
오 채 원	2024002	2005-08-08
김 도 윤	2024003	2005-12-01

(10개 행)

[그림 22] 1-3 실행 결과

문항 2 — COALESCE / CASE WHEN / 날짜 함수 활용

2-1. 성적 미입력(NULL) 건을 '미응시'로 표시 — COALESCE

COALESCE는 첫 번째로 나오는 NULL이 아닌 값을 반환한다. score가 NULL인 2건이 '미응시'로 정확히 치환된 것을 확인할 수 있다.

```
university=# SELECT s.name, c.title,
COALESCE(e.score::TEXT, '미응시 ') AS score_display
FROM enrollments e
JOIN students s ON s.id = e.student_id
JOIN courses c ON c.id = e.course_id
ORDER BY s.name;
```

name	title	score_display
강 민 재	유 기 화 학	82.00
김 도 윤	심 리 학 개 론	미 응 시
김 하 늘	자 료 구 조	95.00
김 하 늘	데 이 터 베 이 스	88.00
박 서 준	자 료 구 조	78.00
박 서 준	운 영 체 제	미 응 시
오 채 원	영 어 작 문	91.00
윤 아 린	생 산 관 리	76.00
이 도 현	운 영 체 제	84.00
이 도 현	데 이 터 베 이 스	92.00
이 수 아	미 적 분 학	95.00
정 예 은	열 역 학	90.00
최 지 우	회 로 이론	85.00
한 지 호	미 시 경 제 학	88.00

(14개 행)

[그림 23] 2-1 실행 결과

2-2. 점수 구간별 학점(A~F) 부여 — CASE WHEN

```

university=# SELECT s.name, c.title, e.score,
CASE
    WHEN e.score IS NULL THEN '미응시'
    WHEN e.score >= 90 THEN 'A'
    WHEN e.score >= 80 THEN 'B'
    WHEN e.score >= 70 THEN 'C'
    ELSE 'F'
END AS grade_letter
FROM enrollments e
JOIN students s ON s.id = e.student_id
JOIN courses c ON c.id = e.course_id
ORDER BY e.score DESC NULLS LAST;

```

name	title	score	grade_letter
김하늘	자료구조	95.00	A
이수아	미적분학	95.00	A
이도현	데이터베이스	92.00	A
오채원	영어작문	91.00	A
정예은	열역학	90.00	A
김하늘	데이터베이스	88.00	B
한지호	미시경제학	88.00	B
최지우	회로이론	85.00	B
이도현	운영체제	84.00	B
강민재	유기화학	82.00	B
박서준	자료구조	78.00	C
윤아린	생산관리	76.00	C
박서준	운영체제		미응시
김도윤	심리학개론		미응시

(14개 행)

[그림 24] 2-2 실행 결과 (미응시 건은 NULLS LAST로 맨 뒤에 정렬됨)

2-3. 교수 재직 연차 계산 — 날짜 함수 (AGE, EXTRACT)

```

university=# SELECT name, hired_date,
EXTRACT(YEAR FROM AGE(now(), hired_date)) AS years_employed
FROM professors
ORDER BY years_employed DESC;

```

name	hired_date	years_employed
윤서아	2011-03-01	15
박지훈	2012-03-01	14
한소율	2013-03-01	13
김민준	2015-03-01	11
강도윤	2014-09-01	11
최유진	2016-03-01	10
임재현	2017-03-01	9
이서연	2018-09-01	7
정하은	2019-03-01	7
오민서	2020-03-01	6

(10개 행)

[그림 25] 2-3 실행 결과

문항 3 — 수강신청 교차 테이블 JOIN 조회

3-1. 학생-수강신청-강좌-교수 4 개 테이블 JOIN

담당교수가 NULL인 강좌(심리학개론)가 있으므로, professors 테이블과의 연결은 LEFT JOIN을 사용했다.

```
university=# SELECT s.name AS student_name,  
마크업 도구 막대 보기 AS course_title,  
p.name AS professor_name,  
e.score  
FROM enrollments e  
JOIN students s ON s.id = e.student_id  
JOIN courses c ON c.id = e.course_id  
LEFT JOIN professors p ON p.id = c.professor_id  
ORDER BY s.name, c.title;
```

student_name	course_title	professor_name	score
강민재	유기화학	정하은	82.00
김도윤	심리학개론		
김하늘	데이터베이스	이서연	88.00
김하늘	자료구조	김민준	95.00
박서준	운영체제	김민준	
박서준	자료구조	김민준	78.00
오채원	영어작문	한소율	91.00
윤아린	생산관리	강도윤	76.00
이도현	데이터베이스	이서연	92.00
이도현	운영체제	김민준	84.00
이수아	미적분학	오민서	95.00
정예은	열역학	최유진	90.00
최지우	회로이론	박지훈	85.00
한지호	미시경제학	임재현	88.00

(14개 행)

[그림 26] 3-1 실행 결과

3-2. 한 번도 수강 신청하지 않은 학생 찾기 — Anti-Join (NOT EXISTS)

NOT IN 대신 NOT EXISTS를 사용해 NULL 함정 없이 안전하게 조회했다.

```
university=# SELECT s.name, s.student_no  
FROM students s  
WHERE NOT EXISTS (  
SELECT 1 FROM enrollments e WHERE e.student_id = s.id  
);
```

name	student_no
임서윤	2023003

(1개 행)

[그림 27] 3-2 실행 결과

8. 보너스 쿼리 및 제약조건 동작 검증

필수 문항 외에 수업에서 배운 Window Function, GROUP BY+HAVING을 추가로 실습했고, 설계한 FK 제약조건 3종(SET NULL/RESTRICT/CASCADE)이 실제로 의도대로 동작하는지 DELETE 명령으로 직접 검증했다.

8-1. Window Function — 학과별 성적 순위

GROUP BY와 달리 개별 행을 유지하면서 RANK()로 학과별 순위를 매겼다.

```
university=# SELECT d.name AS department_name, s.name AS student_name, e.score,
    RANK() OVER (PARTITION BY d.id ORDER BY e.score DESC NULLS LAST) AS rank_in_dept
FROM enrollments e
JOIN students s ON s.id = e.student_id
JOIN departments d ON d.id = s.department_id
ORDER BY d.name, rank_in_dept;
```

department_name	student_name	score	rank_in_dept
경제학과	한지호	88.00	1
기계공학과	정예은	90.00	1
산업공학과	윤아린	76.00	1
수학과	이수아	95.00	1
심리학과	김도윤		1
영어영문학과	오채원	91.00	1
전자공학과	최지우	85.00	1
컴퓨터공학과	김하늘	95.00	1
컴퓨터공학과	이도현	92.00	2
컴퓨터공학과	김하늘	88.00	3
컴퓨터공학과	이도현	84.00	4
컴퓨터공학과	박서준	78.00	5
컴퓨터공학과	박서준		6
화학공학과	강민재	82.00	1

(14개 행)

[그림 28] 학과별 성적 순위 (RANK OVER PARTITION BY)

8-2. GROUP BY + HAVING — 학과별 평균 성적 집계

평균 성적 80점 이상인 학과만 필터링했다. AVG는 NULL을 자동으로 제외하므로 미응시 2건은 평균 계산에서 자동으로 빠졌다.

```
university=# SELECT d.name AS department_name,
    COUNT(e.student_id) AS enrollment_count,
    ROUND(AVG(e.score), 1) AS avg_score
FROM enrollments e
JOIN students s ON s.id = e.student_id
JOIN departments d ON d.id = s.department_id
GROUP BY d.id, d.name
HAVING AVG(e.score) >= 80
ORDER BY avg_score DESC;
```

department_name	enrollment_count	avg_score
수학과	1	95.0
영어영문학과	1	91.0
기계공학과	1	90.0
경제학과	1	88.0
컴퓨터공학과	6	87.4
전자공학과	1	85.0
화학공학과	1	82.0

(7개 행)

[그림 29] 학과별 평균 성적 80점 이상 집계

8-3. 제약조건 검증

검증 1 — RESTRICT: 재학생이 있는 학과는 삭제되지 않아야 한다

```
DELETE FROM departments WHERE code = 'CSE';
```

```
[university=# DELETE FROM departments WHERE code = 'CSE';
ERROR: update or delete on table "departments" violates foreign key constraint "students_department_id_fkey" on table "
students"
상세 정보: Key (id)=(1) is still referenced from table "students".
```

[그림 30] RESTRICT 동작 확인 — 외래키 제약 위반 에러 발생

예상대로 컴퓨터공학과(CSE)에 재학생이 있어 삭제가 차단되고 외래키 제약 위반 에러가 발생했다. 설계 의도가 정확히 구현되었음을 확인했다.

검증 2 — SET NULL: 교수를 삭제해도 강좌는 남고 professor_id 만 NULL 이 된다

```
BEGIN;
DELETE FROM professors WHERE email = 'minjun.kim@univ.ac.kr';
SELECT code, title, professor_id FROM courses WHERE code IN ('CSE201','CSE301');
ROLLBACK;
```

```
[university=# BEGIN;
BEGIN
university=# DELETE FROM professors WHERE email = 'minjun.kim@univ.ac.kr';
[SELECT code, title, professor_id FROM courses WHERE code IN ('CSE201','CSE301');
DELETE 1
  code | title | professor_id
-----+-----+-----
  CSE201 | 자료 구조 | 
  CSE301 | 운영 체제 | 
(2개 행)

[university=# ROLLBACK;
ROLLBACK
```

[그림 31] SET NULL 동작 확인 — 강좌는 유지되고 professor_id가 NULL로 변경됨

교수를 삭제한 뒤에도 자료구조·운영체제 강좌 2건이 삭제되지 않고 남아있으며 professor_id만 NULL로 바뀌었다. 검증 후 ROLLBACK으로 실제 데이터는 원상복구했다.

검증 3 — CASCADE: 학생을 삭제하면 수강신청 기록도 함께 삭제된다

```
BEGIN;
SELECT COUNT(*) FROM enrollments WHERE student_id = (SELECT id FROM students WHERE student_no='2021001');
DELETE FROM students WHERE student_no = '2021001';
SELECT COUNT(*) FROM enrollments WHERE student_id = (SELECT id FROM students WHERE student_no='2021001');
ROLLBACK;
```

```

(university=# BEGIN;
BEGIN
(university=# SELECT COUNT(*) FROM enrollments WHERE student_id = (SELECT id FROM students WHERE student_no='2021001');
count
-----
2
(1개 행)

(university=# DELETE FROM students WHERE student_no = '2021001';
SELECT COUNT(*) FROM enrollments WHERE student_id = (SELECT id FROM students WHERE student_no='2021001');
DELETE 1
count
-----
0
(1개 행)

(university=# ROLLBACK;
ROLLBACK

```

[그림 32] CASCADE 동작 확인 — 삭제 전 2건 → 삭제 후 0건

학생 삭제 전 수강신청 2건이 삭제 후 0건으로 줄어든 것을 확인했다. 약한 엔티티인 enrollments가 부모(students)에 완전히 종속되어 함께 삭제됨을 실증했다. 트랜잭션(BEGIN~ROLLBACK)으로 감싸 실제 데이터는 보존했다.

9. 결론 및 회고

이번 실습을 하면서 가장 크게 느낀 건, ERD를 먼저 대충 그리고 시작했을 때랑 개념→논리→물리 단계를 나눠서 하나씩 확정하고 넘어갔을 때의 결과물 차이가 생각보다 크다는 점이였다. 처음엔 그냥 테이블 5개 그리고 바로 DDL 짜면 되지 않을까 싶었는데, 막상 "왜 이 컬럼을 이 타입으로 했는지", "왜 이 관계는 CASCADE고 저건 SET NULL인지"를 한 단계씩 정리하면서 짜니까 나중에 DDL을 쓸 때 해매는 게 거의 없었다. 특히 FK 삭제 옵션을 세 가지(SET NULL/RESTRICT/CASCADE)로 일부러 다르게 설계해본 게 제일 기억에 남는다. 직접 세 개를 다 써보고 DELETE 문으로 실제로 에러가 나는지, NULL로 바뀌는지, 같이 삭제되는지를 눈으로 확인하니까 훨씬 확실하게 이해됐다.

실습 문항에는 없었지만 Window Function이랑 GROUP BY + HAVING을 추가로 써본 것도 도움이 됐다. 수업 시간엔 개념으로만 배웠던 걸 내가 만든 데이터로 직접 돌려보니, RANK()가 GROUP BY랑 뭐가 다른지 체감이 됐다.

아쉬웠던 부분은 psql 명령어를 몇 번 실수했다는 점이다. 큰 문제는 아니었지만 터미널 작업은 사소한 오타 하나가 다음 명령까지 영향을 준다는 걸 다시 느꼈다. 다음에 비슷한 실습을 하면 psql 명령어를 미리 스크립트 파일로 정리해두고 wii로 실행하는 방식을 써보고 싶다.

전체적으로 ERD 설계 → DDL → DML → 쿼리 → 제약조건 검증까지 한 번에 쭉 이어서 해보니, 각 단계가 왜 그 순서로 진행되어야 하는지가 이해됐다. 특히 "설계한 대로 실제로 동작하는지 검증"하는 과정이 있어야 설계가 완성된다는 걸 배웠다.